

⚠ Please do not reach out to reviewers directly; messaging them will not speed up your review.

Understanding Milestones

In Terminal Bench Edition 2, we are moving beyond "all-or-nothing" task completion. To better evaluate the reasoning traces of frontier agents, a subset of tasks now incorporates **Milestones**.

Terminology note: The Harbor framework refers to these as **multi-step tasks** ([Harbor docs](#)). We use the term **Milestones** throughout our documentation for consistency, but the two are interchangeable — a "milestone" is the same thing as a "step."

What are Milestones?

Milestones divide a complex, multi-step engineering task into standalone, sequential subtasks. Instead of the agent wandering blindly toward a final state, Milestones provide a structured path that allows the Harbor framework to assign incremental rewards and validate the agent's process.

The Milestone Rule

A Milestone task must be designed so that each stage is a prerequisite for the next. The agent works through the milestones in order, and each milestone is verified independently before the next one runs.

How Milestone Tasks are Structured

Milestone tasks use a `steps/` directory at the task root, where each milestone is a self-contained subdirectory with its own instruction, tests, and oracle solution. This is a meaningful change from non-milestone tasks, which keep `instruction.md`, `tests/`, and `solution/` at the task root.

Copy

```

your-task/
├── task.toml                # Includes [[steps]] blocks (see
├── environment/
│   ├── Dockerfile
│   └── ...                  # Any environment files
└── steps/
    ├── milestone_1/
    │   ├── instruction.md   # Prompt the agent sees for mil
    │   ├── tests/
    │   │   ├── test.sh      # Test runner that produces rew
    │   │   └── test_m1.py   # Pytest assertions (TestMilest
    │   └── solution/
    │       ├── solve.sh     # Wrapper – calls solve1.sh
    │       └── solve1.sh    # Oracle solution for milestone
    ├── milestone_2/
    │   └── ...              # Same structure
    └── milestone_3/
        └── ...

```

Files persist across milestones. The container filesystem is shared between milestones, so files left behind by milestone 1 (whether by the agent or by `solve1.sh`) are visible to milestone 2 and beyond. This is intentional — it's how milestones build on each other — but means you should be careful not to clobber files an earlier milestone created.

Milestone Components

To implement milestones correctly in your task, you must align across each of these components:

1. Per-Milestone Instructions

(`steps/milestone_N/instruction.md`)

Each milestone has its own `instruction.md` containing only that milestone's prompt. The agent sees only the current milestone's instruction when working on it.

- Milestone 1's `instruction.md` should include any overall task context the agent needs to get oriented (since it's the first thing the agent sees).
- Subsequent milestones can be shorter — they describe only the new requirements for that milestone, building on what's already done.
- **Do not include a top-level `instruction.md` at the task root.** It will be ignored.

2. Per-Milestone Verifiers (`steps/milestone_N/tests/`)

Each milestone has its own `tests/` directory with:

- `test.sh` — the test runner. Runs pytest on the milestone's test file and writes `1` or `0` to `/logs/verifier/reward.txt` based on the result.
- `test_mN.py` — pytest assertions for milestone `N`, organized in a single `TestMilestoneN` class.

Each milestone's tests must:

- Score only that milestone's completion (not future milestones).
- Be deterministic (pass/fail).
- Tolerate any state left behind by previous milestones.

3. Per-Milestone Oracle Solutions

(`steps/milestone_N/solution/`)

Each milestone has its own `solution/` directory with:

- `solve.sh` — a small wrapper that runs the milestone's oracle script.
- `solveN.sh` — the actual oracle solution containing only the commands required for milestone `N`.

The wrapper pattern looks like this:

BASH

 Copy

```
#!/bin/bash
set -euo pipefail
SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
bash "$SCRIPT_DIR/solve1.sh"
```

4. Metadata (`task.toml`)

Milestone tasks add `[[steps]]` array-of-tables entries to `task.toml`, one per milestone. The order in the file determines execution order.

TOML

 Copy

```
# Task configuration schema version
version = "2.0"

[metadata]
author_name = "your-name"
author_email = "your-email"
difficulty = "hard"
category = "software-engineering"
subcategories = []
# Total milestone count – must match the number of [[steps]] blocks
number_of_milestones = 2
codebase_size = "minimal"
languages = ["python", "bash"]
tags = ["..."]
expert_time_estimate_min = 60
junior_time_estimate_min = 120

# Sandbox environment limits – applied across all milestones
[environment]
build_timeout_sec = 600.0
cpus = 2
memory_mb = 4096
storage_mb = 10240
workdir = "/app"

# Milestone 1
[[steps]]
name = "milestone_1"
```

```

[steps.agent]
timeout_sec = 1200.0

[steps.verifier]
timeout_sec = 450.0

# Milestone 2
[[steps]]
name = "milestone_2"

[steps.agent]
timeout_sec = 1200.0

[steps.verifier]
timeout_sec = 450.0

```

Notes on `task.toml`:

- `number_of_milestones` in `[metadata]` must equal the number of `[[steps]]` blocks. Set to `0` for non-milestone tasks.
- `[[steps]].name` must be `milestone_1`, `milestone_2`, etc. (snake_case, sequential) and must match the directory name under `steps/`.
- **No top-level `[verifier]` or `[agent]` blocks** — those are replaced by per-milestone `[steps.verifier]` and `[steps.agent]`.
- `[environment]` still applies globally — the same container is shared across all milestones.

5. Rubric

Your rubric should cover aspects of all your task's milestones.

Rubric points per milestone: In a milestone task, each milestone should be represented in the rubric with **10–40 points** allocated to that milestone.

The 10–40 points refers to the **maximum cumulative score, calculated by summing all positive criteria**. This scales per milestone:

- 1 milestone → 10–40 pts
- 2 milestones → 20–80 pts

- 3 milestones → 30–120 pts

For more than three milestones, the same pattern applies: total points fall between **(number of milestones × 10)** and **(number of milestones × 40)**.

Best Practices for Milestones

- **No "Leaky" Milestones:** Ensure milestone 2 cannot pass if milestone 1 fails. Each milestone must depend on the previous one being correctly completed.
- **Clear Boundaries:** An agent should know exactly when a milestone is finished based on its own `instruction.md`.
- **Balance:** Avoid over-segmenting. Most milestone tasks should have between 2 and 5 milestones. If a task seems to require 10 milestones, combine the closely-related ones or focus on the most challenging parts.
- **Mind the shared filesystem:** Files persist across milestones. If milestone 2 needs a clean state for some path, have its `solve2.sh` reset it explicitly rather than assuming.
- **Don't put files at the task root:** No top-level `instruction.md`, `tests/`, `solution/`, or `milestone_x.md` — everything per-milestone lives under `steps/milestone_N/`.

PREVIOUS

[NEW: Submission Diversity Requirements](#)

NEXT

[NEW: Task Sub-types](#)